1 文档说明

本文件是音赴光年 V0.1.0 的设计说明书，用作该软件著作权登记的文档鉴别材料。说明书说明这套程序做什么、分成哪些部分、数据放在哪里、播放和歌词如何衔接。文中的模块名称、数据表和界面分档与同期源程序一致。源程序用 Dart 编写，界面用 Flutter 组织，同一套代码运行在 Windows、macOS、Linux、Android 和 iOS 上。

2 软件概述

音赴光年是一款音频播放软件，英文名 Soundward Lightyear。它播放音乐视频里的声音，不播放画面。用户可以按歌名或歌手搜索，从结果里点选曲目后开始听。界面上的标题整理成歌名和歌手。歌词单独显示，并跟着当前进度往下走。一首曲目听过之后把音频留在本机，再次播放时直接读本地文件。没有网络时，软件不再拉取新的内容，只从已经留下的曲目里继续播放。

歌单保存在当前这台设备上。用户可以新建歌单、改名、删除、调整曲目顺序，也可以把当前队列整列存成一张歌单。搜索、播放、看歌词和编辑歌单不要求登录。登录之后才能使用更高的音质档。登录凭据只放在本机的系统安全存储里。

3 开发目的与运行环境

开发目的是在电脑和手机上收听音频，并完成搜索、歌词、本地歌单和离线续播。面向领域是音乐。开发使用的操作系统是 Windows 11，开发工具是 Flutter。运行平台包括 Windows、Android、macOS、iOS 和 Linux。运行时依赖 Flutter 运行环境。编程语言是 Dart。

开发所用的硬件是普通个人计算机，处理器 2GHz 及以上，内存 8GB 及以上。运行时的硬件是个人计算机或手机，内存 4GB 及以上。软件不在界面中解码视频画面，后台播放只处理音频。

4 总体结构

源程序放在 lib 目录，按职责分成五层。main.dart 是进程入口。app 目录放置应用外壳，包括主题、路由、桌面窗口和按窗口尺寸决定的布局。core 目录放置日志、文件路径和安全存储。domain 目录放置播放队列、标题清洗和歌词评分，这些规则不依赖具体页面。data 目录放置网络访问、SQLite 数据库、音频缓存和系统媒体会话。features 目录放置搜索、播放、歌单、队列、歌词和设置页面。

页面通过仓储接口读取搜索结果和音频地址，不直接拼接网络请求。播放顺序由 PlaybackQueue 决定，不把随机顺序交给播放器组件内部生成。这样，切到随机、再切回顺序时，可以回到原来的排列，并停在当前这首上。

5 启动流程

进程从 main 进入。启动时先接住界面构建错误和未捕获异常，写入日志，避免页面构建失败时没有任何记录。桌面端接着初始化无边框窗口。若本机已经有一个实例在运行，新的实例不另开一份界面，而是通知已有实例把主窗口显示出来并放到前台。通道名称是 bili_audio/single_instance，调用的方法是 showMainWindow。

窗口就绪后完成依赖装配：准备数据库、缓存目录、播放器和歌词管理器，再把界面交给 BiliAudioApp。用户上次留下的队列、播放模式和进度会在打开后恢复，但不会自动开始出声，需要用户再次按下播放。若启动过程本身失败，入口显示单独的错误页，而不是空白窗口。

6 窗口与布局

布局只看窗口的逻辑像素宽高，不按手机、平板或电脑的设备类型写死。宽度小于 600 为 compact。600 至 840 为 medium。840 至 1194 为 expanded。1194 至 1600 为 large。1600 以上为 extraLarge。高度小于 500 为 short。500 至 844 为 regular。844 以上为 tall。

高度不足 500，或者宽度不足 600 时，使用底部导航。底部有三个入口：发现、歌单、我的。正在播放的内容收成底部一条迷你播放条。宽度达到 600 并且高度至少 500 时，导航改到左侧。宽度达到 1194 并且高度至少 500 时，使用左右分栏：左侧是当前页面，右侧常驻正在播放，包括封面、当前一句歌词、进度、控制条和队列。左右之间的分隔条可以拖动。左栏占可用宽度的比例限制在 0.4 到 0.75，松手后记住这个比例。11 英寸平板横屏宽度是 1194，因此落在这套左右分栏里。窗口拉宽或收窄时，按上述档位当场切换，不需要重启。

另有一套车机布局，和上面的三档无关。设置为自动时，只在系统报告车载界面时使用大按钮。也可以始终打开或始终关闭。车机界面左侧是封面和播放控制，右侧在队列和歌词之间切换。

桌面窗口无边框，标题栏上提供最小化、最大化和关闭。关闭窗口时收到系统托盘，不退出进程。托盘菜单可以再次打开窗口。主题可以跟随系统、固定浅色或固定深色，并可以更换一套主题色。

7 发现与搜索

发现页在用户还没有输入关键词时展示音乐推荐。推荐分成两组。一组是排行，包括按近期热度、评价相对更高的曲目，以及原创、翻唱、音乐视频、现场、演奏和 VOCALOID。另一组是年代，曲目来自年代歌单，播放地址再按歌名和歌手查找。推荐可以先把已经拿到的一部分显示出来，不必等整页全部到齐。

搜索按歌名或歌手进行，结果分页加载，并记住搜索历史。时长超过 15 分钟的结果

会被标出并淡化，便于留下能当歌曲听的内容。结果进入界面之前先做标题整理，列表上显示整理后的歌名和歌手。点选一首之后，若该视频有多个分 P，每一 P 作为一首独立曲目进入队列，按顺序衔接，并各自匹配歌词。界面上能看到当前是第几 P、这一 P 的标题，以及它属于哪一个视频。

8 标题整理

视频标题里常夹着音质、官方音乐视频、4K、翻唱、中字这类修饰。TitleCleaner 去掉这类括号修饰，再按分隔符拆出歌名和歌手，界面统一显示为歌名和歌手。原始标题仍然保留，用户可以查看。整理错了可以手改，改过的歌名和歌手会写进曲目记录，下次仍用改过的结果。

标题里带有伴奏、纯音乐或 Instrumental 时，把这首标成无歌词，不再去查找歌词。整理后的歌名和歌手同时用于歌词匹配，减少标题噪声造成的错配。列表、播放页、通知和锁屏使用同一套整理结果。

9 播放队列

播放顺序由 PlaybackQueue 统一管理。模式有五种：顺序播放、列表循环、单曲循环、随机播放和智能随机。顺序播放按队列走到最后一首后停止。列表循环走到最后一首后回到第一首。单曲循环重复当前这首。

随机播放不在进入模式时就把整列打乱。当前这首保持不动。用户点下一首时，才从这一轮还没有听过的曲目里抽一首。一轮之内不重复。这一轮都听过之后重新开始，刚离开的那首不会马上再被抽到。智能随机在此之上尽量换一位歌手：队列里还有其他歌手时，下一首尽量不是同一位。

上一首按实际听过的顺序往回退，退到的是刚才那首，不是列表里的相邻项。从随机切回顺序时，队列恢复成原来的排列，并停在当前这首上。随机模式下不能拖拽改顺序。用户也可以指定下一首，被指定的曲目优先于随机抽取。

播放控制包括开始、暂停、拖动进度、上一首和下一首。倍速可以选 0.75、1、1.25、1.5 和 2。定时停止可以选 15 分钟、30 分钟、60 分钟，也可以在当前这首结束或下一首结束后停下。当前队列可以移除曲目，清空时可以保留正在听的一首，也可以把整列存成歌单。播放失败时跳过当前这首，避免停在无法出声的一项上。

10 音频地址与本地缓存

播放前按曲目和音质档解析音频地址。音质档包括 64K、132K、192K、Dolby 和 Hi-Res。未登录时最高使用 132K。音频地址有时效，过期后在后台重新解析，不要求用户重新点播放。

播放过程中把音频写入本机。同一首再次播放并且本地文件已经完整时，直接读本地

文件，不再请求音频地址。封面、歌词和整理后的歌名、歌手一起留下，离线时仍能看出是哪一首。缓存上限可以在 250MB、500MB、1GB、2GB 和 5GB 里选择。空间不够时，清掉最久没有播放、并且没有放进歌单的曲目。已经放进歌单的曲目标记为保留，不参与这次清理。设置里可以查看已缓存的数量和占用，按首删除，或一次性清掉未加入歌单的缓存。未缓存的曲目仍然需要网络。

11 歌词

歌词按曲目的视频标识和分 P 标识保存，偏移也按这一对标识记住。查找时先读本机已经匹配过的结果。若用户曾经把这首标成无歌词，直接返回无歌词，不再访问外部来源。本机没有记录时，再按各来源的优先级依次查询。

匹配同时看歌名、歌手和时长。得分达到 0.80 时直接采用并写入本机。得分在 0.60 到 0.80 之间时列出候选，由用户挑选。低于 0.60 时不自动套用。同一首的并发查找会合并，避免重复请求。

歌词视图跟着播放进度滚动，点某一行会跳到对应时间。时间偏了可以按 0.1 秒微调。窄窗口里歌词在播放页中滚动。窗口更宽时，封面和歌词可以左右并排。底部迷你条只显示当前一句。桌面分栏的右侧正在播放也显示当前一句。

12 歌单与播放历史

歌单写在本机数据库里，不登录也能用。每张歌单记录名称、封面和曲目顺序。同一视频的同一分 P 在同一张歌单里只保留一次。搜索结果、正在播放的这首和当前队列都可以加入歌单，也可以在选择歌单时当场新建一张。

打开一张歌单后可以按歌单顺序从第一首播放，也可以按随机规则打散后播放。点列表中的某一首，从那一首开始播放整张歌单。曲目可以拖动调整顺序，也可以移除。放进歌单的曲目受到缓存保护。

播放历史和歌单在同一个入口里，用顶部标签分开。历史按今天、昨天、本周和更早分组，可以清空。每条历史记录保存播放时间和播放到的位置。

13 登录

不登录就可以搜索、以基础音质播放、看歌词、使用缓存和编辑本地歌单。需要更高音质，或内容本身要求登录时，再提示登录。登录方式包括扫码和网页。确认登录后得到的凭据交给本机系统安全存储，不写入普通文本设置，也不复制到其他设备。

14 本机数据

业务数据放在 SQLite 数据库中，主要有以下各表。

曲目表 tracks 以视频标识 bvid 和分 P 标识 cid 为主键。其中保存原始标题、整理后的歌名、歌手、用户是否手改过、时长、封面、上传者、分 P 序号和总分 P 数。时

长必须大于 0。

歌单表 playlists 保存名称、封面、是否只读、来源和来源标识。名称长度限制在 1 到 100。歌单与曲目的关联表 playlist_tracks 保存所属歌单、曲目标识和从 0 开始的顺序。删除歌单时，关联行一并删除。

播放历史表 play_history 保存曲目、播放时间、播放到的毫秒位置、完成度和来源。完成度限制在 0 到 1。

歌词缓存表 lyrics_cache 以 bvid 和 cid 为主键，保存本地歌词文件相对路径、来源、匹配得分、用户校准的偏移毫秒，以及是否标成无歌词。

音频缓存索引表 cache_index 以缓存键为主键。缓存键由视频标识、分 P 标识和音质档组成。表中保存相对路径、文件大小、是否完整、已下载字节、最近播放时间，以及是否因在歌单中而保留。已下载字节不能大于文件大小。

音频地址表 stream_urls 按曲目和音质档保存地址、备用地址、媒体类型和过期时间。队列快照表 queue_snapshot 只保留一行，保存队列原文、当前顺序、光标、播放模式、随机种子、已播放记录和进度。设置表 settings 以键值保存音质、主题和缓存上限等偏好。

15 桌面媒体控制与更新

桌面端把当前曲目交给系统媒体会话，系统媒体键可以暂停和切歌。通知和锁屏显示整理后的歌名和歌手。应用重新打开后恢复上次的队列和进度，但不自动播放。

设置里可以检查更新。安装包的构建号高于本机时提示升级。更新说明和安装包来自发布记录。预发布构建不单独当成一个需要提示的正式版本。安装前检查安装包是否对得上当前系统。

16 出错与离线

页面构建失败和未捕获异常写入日志。日志对凭据类字段做脱敏，不把敏感值原样落下。音频地址失效或网络失败时，播放器跳过无法播放的一项并继续队列。仅在本地已有文件时，没有网络也可以播放已缓存的曲目。未缓存的曲目仍要联网。队列、歌单、歌词偏移和主题都写在本机，不依赖这次是否登录。

17 使用说明之搜索与播放

打开软件后进入发现页。尚未输入关键词时，页面展示音乐推荐。推荐里可以切换热度、评价、原创、翻唱、音乐视频、现场、演奏、VOCALOID，也可以按年代查看。点一首即从该首开始播放。若这是一个多分 P 的合集，各分 P 按顺序进入队列，每一 P 单独显示标题并单独匹配歌词。

需要指定曲目时，在搜索框输入歌名或歌手。结果按页继续加载。时长超过 15 分钟

的条目会被标出并淡化。点开一条结果后开始播放，界面标题使用整理后的歌名和歌手。搜索过的词留在本机，下次可以再次点选。

播放页可以暂停、继续、拖动进度、切到上一首或下一首。倍速在 0.75、1、1.25、1.5、2 之间选择。定时停止可以选择 15 分钟、30 分钟、60 分钟，也可以选择播完当前这首或播完下一首后停止。当前队列里可以移除曲目，清空时可以留下正在听的一首，也可以把整列保存为歌单。

播放模式从顺序、列表循环、单曲循环、随机和智能随机里选择。随机时当前这首不动，从下一首开始在本轮还没听过的曲目里抽取。上一首回到刚才实际听过的那首。随机时不能拖动改变队列顺序。从随机改回顺序后，队列回到原来的先后，并停在当前这首。

18 使用说明之歌词、歌单与缓存

播放时软件自动查找歌词。已经匹配过的直接使用。没有记录时再按各来源查找。很像的歌词直接采用。不太确定时列出候选，由用户选择。对不上则不自动填上。点歌词的某一行，播放跳到对应时间。若歌词整体偏早或偏晚，按 0.1 秒调整，这个偏移会按该视频的该分 P 记住。也可以把这首标成无歌词，以后不再查找。标题里已经能看出是伴奏或纯音乐的，不会去查找歌词。

歌单从底部或侧栏的歌单入口进入。可以新建、改名和删除。把搜索结果、正在播放的曲目或当前队列放进歌单时，同一个视频的同一个分 P 不会重复加入。打开一张歌单后，可以按列表顺序全部播放，也可以整张随机播放。点其中一首，则从这一首开始播放这张歌单。曲目可以拖动排序，也可以从歌单中移除。

听过的曲目会写入播放历史。历史和歌单在同一入口，用顶部标签切换，并按今天、昨天、本周和更早分组。历史可以清空。

音频在播放时写入本机。同一首再次播放时，若本地文件已经完整，就不再请求音频地址。设置中可以查看缓存占用，按首删除，或清掉还没有放进歌单的缓存。缓存上限在 250MB、500MB、1GB、2GB、5GB 中选择。已经放进歌单的曲目会留下来。没网时可以继续听已经缓存的曲目。

19 使用说明之账号、外观与更新

不登录即可搜索、播放、看歌词、缓存和编辑歌单。未登录时音质最高到 132K。需要更高音质时再登录。登录可以用扫码或网页完成。登录凭据只放在本机系统安全存储中。

外观可以跟随系统，也可以固定为浅色或深色，并可以更换主题色。车机布局可以设为自动、始终开启或关闭。自动时只在系统处于车载界面时使用大按钮。车机界面一边是封面和播放控制，另一边在队列和歌词之间切换。

桌面端把窗口关闭后，程序留在系统托盘里，媒体键仍可以暂停和切歌。再次打开时

恢复上次的队列、模式和进度，但不会自动出声。设置里可以检查更新。比本机更新的正式版本会提示安装。

把窗口拉窄或拉宽时，导航和播放区域按宽度和高度切换。宽度不足 600，或高度不足 500 时，使用底部三个入口和底部播放条。更宽并且足够高时，导航移到左侧。宽度达到 1194 且高度至少 500 时，右侧一直留着正在播放，中间的分隔可以拖动，左侧大约占四成到七成五。

20 源程序与说明书的对应关系

本说明书后面按源程序文件列出各文件的位置、文件自身的说明、定义的类型，以及可从外部调用的操作。文件顺序与程序鉴别材料里的源程序顺序相同：先是入口，然后是应用外壳、基础能力、领域规则、数据与播放实现，最后是界面。程序鉴别材料取的是这条源程序的前 30 页和后 30 页。页眉上的软件名称和版本号与本说明书相同。

18 程序入口

负责进程启动、错误兜底、桌面单实例，以及把界面交到应用外壳。

文件 lib/main.dart。单实例通道:当第二个实例启动并检测到已有实例运行时,原生层会向本实例发送"showMainWindow"调用。

19 应用外壳

负责主题、路由、窗口、托盘和按窗口尺寸切换布局。

文件 lib/app/adaptive/layout_settings.dart。设置里的车机布局开关。自动只在系统报告车载界面时生效。本文件定义了CarLayoutMode、LayoutSettings、LayoutSettingsNotifier。LayoutSettings：分隔比例与车机开关。车机是否显示还要看systemCar。

文件 lib/app/adaptive/max_width_box.dart。表单和设置在宽窗口上居中，并不超过maxWidth。本文件定义了MaxWidthBox。

文件 lib/app/adaptive/player_shortcuts.dart。桌面和iPad外接键盘。焦点在输入框时不拦截。本文件定义了PlayerShortcutLayer。

文件 lib/app/adaptive/split_view.dart。主从栏分隔。比例是左栏占可用宽度的份额，限制在0.4 – 0.75。本文件定义了SplitView。

文件 lib/app/adaptive/system_car.dart。Android车载UI模式。其他平台、读不到或调用失败时为false。

文件 lib/app/adaptive/window_metrics.dart。窗口宽度档。布局只看逻辑像素，不看设备类型。large从1194起：11英寸iPad横屏是1194，任务要求这一档用主从栏。本文件定义了WindowWidth、WindowHeight、WindowMetrics、WindowMetricsScope。

WindowHeight：窗口高度档。只影响纵向密度，不单独决定导航形态。WindowMetrics：由内容区宽高得到的布局档位。WindowMetricsScope：壳层把内容区档位传给子页面。播放页等壳外路由自行用WindowMetrics.fromSize。

文件 lib/app/app.dart。桌面端窗口外壳(沿用talix_companion_flutter的无边框方案)。窗口背景在main()中设为透明,这里用ClipRRect裁出圆角。顶部标题栏可拖拽。Windows/Linux在右侧放窗体按钮；macOS保留系统红绿灯，标题栏与红绿灯同高，标题靠右并垂直居中。本文件定义了BiliAudioApp。

文件 lib/app/bootstrap.dart。构建真实依赖并返回ProviderScopeoverrides(M6)要求PathManager.init已完成。失败时抛出,由main展示启动错误页。

文件 lib/app/motion/motion.dart。页面切换、状态替换和列表入场共用的时长。系统开启「减少动态效果」时，过渡时长为零，列表不再错峰入场。本文件定义了Motion、TabMotion、TabTransition、PushTransition、MotionSwap、ListEntranceScope、ListEntrance、SkeletonList。TabTransition：底栏切换：旧页先淡出并让开，新页再淡入。中间露出的是壳层底色。PushTransition：压入的全屏页：淡入并带一小段位移，返回时沿原路退出。MotionSwap：加载、空、出错、有数据之间的淡入替换。ListEntranceScope：包住列表。只有出现后很短时间内构建的前几项会入场，滚出再滚回、或加载更多，都不再重播。

文件 lib/app/platform/app_quit.dart。保存窗口位置，去掉托盘图标，然后结束桌面端进程。手机端直接返回。托盘还没建好时仍会退出。

文件 lib/app/platform/desktop_controls.dart。托盘、任务栏和程序坞共用的播放快照。本文件定义了DesktopTrackSnapshot、DesktopTaskbarPhase、DesktopCommand、DesktopControlLabels、DesktopMenuEntry。DesktopTaskbarPhase：任务栏图标上的进度样式。

文件 lib/app/platform/desktop_platform.dart。SMTC是否已成功初始化;失败时WindowsMediaBridge不应创建。本文件定义了WindowsMediaBridge。WindowsMediaBridge：WindowsSMTC桥接(M4.3)监听BiliAudioHandler状态并同步到Windows任务栏/媒体键控制。

文件 lib/app/platform/desktop_shell.dart。托盘、关闭收进托盘，以及还没有曲目时的空菜单。

文件 lib/app/platform/open_url.dart。与介绍页、应用商店填写的地址是同一份。

文件 lib/app/platform/window_frame.dart。全屏时桌面壳去掉圆角。本文件定义了WindowFrame、WindowFrameStore。WindowFrame：判断保存的窗口矩形能否用。屏幕列表为空时只校验尺寸。

文件 lib/app/platform/windows_taskbar_host.dart。悬停任务栏图标时的上一首/播放/下一首，以及预览提示。

文件 lib/app/providers.dart。全局依赖(M6)默认实现直接抛错:真实依赖在`bootstrap()`中构建并override,测试中用Fake仓储+Fake引擎override,UI只依赖接口。本文件定义了PlaybackPrefs。PlaybackPrefs：播放器在每次加载曲目时读取的偏好(设置页/登录后修改)。

文件 lib/app/router.dart。应用路由(M0.4/M6)底部导航三个Tab共用HomeShell(迷你播放栏常驻);播放页、队列、歌单详情、缓存管理为全屏页面。本文件定义了NavLogObserver、NavErrorPage。NavLogObserver：记录页面push/pop/replace。弹窗和底部菜单不是PageRoute，不记。NavErrorPage：路由匹配失败或参数非法时的页面，并写入导航错误日志。

文件 lib/app/system_bars.dart。Android/iOS才把内容铺到系统栏下面。桌面和网页保持原样。

文件 lib/app/theme.dart。预设主题种子色(dev-tasksM0.4)。首项为B站粉。本文件定义了ThemeState、ThemeController。ThemeController：主题状态(M0.4):修改后立即生效并持久化,重启后保持。

文件 lib/app/widgets/region_surface.dart。阴影落在相邻区域的哪一侧。本文件定义了RegionEdge、RegionSurface。RegionSurface：功能区与画布同色，只在交界投一层浅阴影。

文件 lib/app/widgets/window_title_bar.dart。自定义窗口标题栏。拖拽移动窗口，双击最大化或还原。Windows/Linux高48，左侧放logo与标题，右侧放最小化、最大化、关闭（关闭为隐藏到托盘）。macOS使用系统红绿灯，不画这组按钮，也不画logo，只把标题靠右；高度与系统标题条一致（28），红绿灯的垂直中心在距顶14。本文件定义了WindowTitleBar。

20 基础能力

负责日志、路径、安全存储和与具体页面无关的工具。

文件 lib/core/logging/app_logger.dart。日志等级。低于AppLogger.minLevel的记录会被丢弃。本文件定义了LogLevel、AppLogger。AppLogger：本地日志。控制台走debugPrint；桌面端同时按天写入init指定的目录（`app-YYYYMMDD.log`），并清理7天前的文件。未调用init时只输出到控制台。Cookie、SESSDATA等密钥不要写进message，网络层用formatQuery/summarizeBody做脱敏。

文件 lib/core/network/bili_error_mapper.dart。B站业务错误码(M2.1)。本文件

定义了BiliCode。

文件 lib/core/network/bili_http.dart。BilibiliHTTP客户端(M2.1,M2.7)拦截器链:Header→Cookie→Wbi→RateLimit→Retry→RiskControl→ErrorMapper。本文件定义了BiliHttp。

文件 lib/core/network/buvid_manager.dart。调用`/x/frontend/finger/spi`的回调,返回(b_3,b_4)。本文件定义了BuvidManager。BuvidManager：buvid3/buvid4设备标识(M2.1)优先读持久化值;否则请求接口;接口失败时本地随机生成兜底。

文件 lib/core/network/interceptors/cookie_interceptor.dart。Cookie拦截器(M2.1):注入登录Cookie+buvid3/4。本文件定义了CookieInterceptor。

文件 lib/core/network/interceptors/error_mapper_interceptor.dart。错误映射拦截器(M2.1):将Dio异常与B站业务码映射为AppError。本文件定义了ErrorMapperInterceptor。

文件 lib/core/network/interceptors/header_interceptor.dart。请求头拦截器(M2.1):User-Agent/Referer/Origin。本文件定义了HeaderInterceptor。

文件 lib/core/network/interceptors/http_log_interceptor.dart。记录B站请求的路径、脱敏查询参数、耗时和响应摘要。不记录Cookie/SESSDATA，也不展开播放地址。本文件定义了HttpLogInterceptor。

文件 lib/core/network/interceptors/rate_limit_interceptor.dart。令牌桶限流(M2.1):默认5req/s。本文件定义了RateLimitInterceptor。

文件 lib/core/network/interceptors/retry_interceptor.dart。重试拦截器(M2.1):网络错误指数退避，最多3次。本文件定义了RetryInterceptor。

文件 lib/core/network/interceptors/risk_control_interceptor.dart。风控拦截器(M2.7)-请求extra`risk_guarded:true`时,冷却期内直接拒绝-收到-412时计数;对`risk_retry:true`的请求退避backoff后重试一次。本文件定义了RiskControlInterceptor。

文件 lib/core/network/interceptors/wbi_interceptor.dart。WBI签名拦截器(M2.1)请求extra中设置`wbi:true`时触发签名;-352/-403时刷新key并重试一次。本文件定义了WbiInterceptor。

文件 lib/core/network/risk_control_guard.dart。风控冷却(M2.7)window内累计threshold次-412进入冷却cooldown;冷却期间受保护的请求(如搜索)直接失败,不再打到服务端。已缓存内容的播放不经过网络,不受影响。本文件定义了RiskControlGuard。

文件 lib/core/network/wbi.dart。WBI签名纯函数(M2.1)算法参考

bilibili-API-collect:`img_key+sub_key`按混淆表重排取前32位得到mixin_key;参数追加`wts`后按key排序、过滤`!'()*`、URL编码,`w_rid=md5(query+mixin_key)`。本文件定义了Wbi。

文件 lib/core/network/wbi_key_manager.dart。拉取`wbi_img`的回调,返回(img_url,sub_url)。本文件定义了WbiKeyManager。WbiKeyManager：WBImixin_key缓存(M2.1)key缓存24h;并发调用合并为一次请求;签名失败时由调用方invalidate。

文件 lib/core/storage/path_manager.dart。统一管理本地文件目录:音频缓存/封面/歌词/数据库。启动时调用一次init,之后通过静态getter同步访问。缓存根目录后续可由设置项覆盖(见dev-tasksM3.5)。本文件定义了PathManager。

文件 lib/core/storage/secure_store.dart。敏感数据存储(M0.3)只暴露Cookie与设备标识的读写,便于在测试中替换为MemorySecureStore。本文件定义了SecureStore、FlutterSecureStore、MemorySecureStore。

文件 lib/core/utils/cover_url.dart。把封面地址收成系统媒体控件能接受的绝对URI。B站接口经常返回`//i0.hdslb.com/...`。这种协议相对地址`Uri.tryParse`能过，但WindowsSMTC会把它当成非法URI并panic。

文件 lib/core/utils/result.dart。Result类型(M1.1)用于表示可能失败的操作结果，替代抛异常。本文件定义了Result。

21 领域规则

负责播放顺序、标题整理、歌词评分等不依赖界面的规则。

文件 lib/domain/album/album_provider.dart。专辑资料查询条件。字段和歌词查询对齐，方便用同一套歌名清洗。本文件定义了AlbumQuery、AlbumCandidate、AlbumSourceUnavailable、AlbumProvider。AlbumCandidate：一个源返回的专辑图、年份和介绍。AlbumSourceUnavailable：专辑源连不上或读响应超时。管理器会重试一次，再换下一个源。AlbumProvider：专辑图和介绍的来源。数字越小越优先。图、年份、介绍可以来自不同的源：当前源缺了哪一项，就继续问后面的源。

文件 lib/domain/cache/lru_policy.dart。缓存条目(M1.7)。本文件定义了CacheEntry、EvictionResult、LruPolicy。EvictionResult：淘汰结果(M1.7)。LruPolicy：LRU淘汰策略(M1.7,纯算法)。

文件 lib/domain/entities/app_error.dart。应用错误类型(M1.1)用于统一错误处理和用户提示。本文件定义了AppError。

文件 lib/domain/entities/audio_track.dart。Track实体(M1.1)对应B站视频的单个分P，包含播放所需的最小信息。不包含流地址(由StreamUrlResolver动态解析)和歌词

(由LyricsManager管理)。本文件定义了Track、AudioQuality。AudioQuality：音质档位(M1.1)。

文件 lib/domain/entities/lyrics.dart。歌词行(M1.1)。本文件定义了LyricLine、Lyrics。

文件 lib/domain/entities/play_history.dart。播放历史记录(M1.1)每次播放达到一定阈值(如播放时长超过30s或进度>50%)后记录一条。用于"最近播放"歌单和统计分析(v0.2.4)。本文件定义了PlayHistory。

文件 lib/domain/entities/playlist.dart。歌单(M1.1)本地创建的播放列表，对应dev-tasks中的"本地歌单"。数据库存储在playlists表，track关联存在playlist_tracks表。本文件定义了Playlist。

文件 lib/domain/entities/stream_url.dart。音频流地址(M1.1)B站音频流地址有时效性(通常2-4小时)，由StreamUrlResolver动态解析和缓存。数据库存储在stream_urls表(bvid,cid,quality联合主键)。本文件定义了StreamUrl。

文件 lib/domain/entities/video.dart。视频分P(M1.1)。本文件定义了VideoPage、Video。

文件 lib/domain/lyrics/lrc_parser.dart。LRC解析器(M1.4)支持:-mm:ssmm:ss.xxmm:ss.xxx格式-单行多时间戳-offset:+/-ms偏移量-元数据标签(ar:ti:al:等)。本文件定义了LrcParser、LyricsTimeline。LyricsTimeline：歌词时间轴(M1.4)用于播放时快速定位当前歌词行。

文件 lib/domain/lyrics/lyrics_provider.dart。歌词查询条件(M5.1)。本文件定义了LyricsQuery、LyricsCandidate、LyricsSourceUnavailable、LyricsProvider。LyricsCandidate：歌词候选(M5.1)。LyricsSourceUnavailable：歌词源连不上或读响应超时。管理器会重试一次，再换下一个歌词源。LyricsProvider：歌词提供者(M5.1)数字越小越优先;按优先级依次查询,首个得分≥自动阈值的候选即停止。

文件 lib/domain/lyrics/lyrics_scorer.dart。歌词相似度评分器(M1.5)用于评估搜索到的歌词是否匹配当前曲目。歌名和歌手走FuzzyMatcher。本文件定义了LyricsScorer、MatchDecision、LyricsMatch。MatchDecision：匹配决策(M1.5)。LyricsMatch：歌词匹配结果(M1.5)。

文件 lib/domain/queue/playback_queue.dart。播放模式(M1.6)。本文件定义了PlayMode、PlaybackQueue。PlaybackQueue：播放队列(M1.6)管理播放顺序、模式切换、随机逻辑。随机播放不预先打乱列表:点下一首时才在本轮未播曲目里选取。支持持久化和恢复。

文件 lib/domain/queue/queue_search.dart。超过这个长度时，拼音索引放到后台建，避免卡住输入。本文件定义了QueueSearchHaystack、QueueSearchIndex。QueueSearchHaystack：一条曲目预先算好的搜索文本。QueueSearchIndex：当前播放顺序的搜索索引。字面量可立即用，拼音可以稍后补上。

文件 lib/domain/text_normalization.dart。文本归一化工具(M1.2)用于标题清洗和歌词匹配的文本预处理。

文件 lib/domain/title/fuzzy_match.dart。模糊匹配。先丢掉分P序号、音质和版本尾巴，再比较剩下的歌名或歌手。`0065.洋葱`和`洋葱`、`Eric周兴哲`和`周兴哲`视为同一段文字。本文件定义了FuzzyMatcher。

文件 lib/domain/title/title_cleaner.dart。标题清洗器(M1.3)将B站视频标题清洗为歌名和歌手，支持规则配置。本文件定义了TitleCleaner。

文件 lib/domain/title/title_rule.dart。标题清洗规则配置(M1.3)。本文件定义了TitleRuleConfig、TitleRule、CleanedTitle。

文件 lib/domain/update/app_version.dart。应用版本号，对应pubspec的`major.minor.patch`和可选的`+build`。远程标签可以写成`v1.2.3`或`1.2.3`。两边都带构建号时才比较构建号；只有一边有构建号时，只比较主版本、次版本和修订号。本文件定义了AppVersion。

文件 lib/domain/update/distribution.dart。是否打成MicrosoftStore包。商店负责更新。构建时传入`--dart-define=MICROSOFT_STORE=true`。

文件 lib/domain/update/installer_asset.dart。升级读取的GitHub仓库，`owner/name`。本文件定义了InstallerKind。InstallerKind：安装包打开方式。

22 数据与播放实现

负责搜索、音频地址、数据库、缓存、歌词来源和系统媒体会话。

文件 lib/data/album/album_http.dart。先抽出歌名和歌手，再拼实际发给远端的关键词。

文件 lib/data/album/album_manager.dart。专辑查询结果。本文件定义了AlbumResult、AlbumFound、AlbumNone、AlbumManager。AlbumFound：已匹配到官方专辑图、年份或介绍。缺的字段留空。AlbumNone：各源都对不上，或上次已经确认没有资料。AlbumManager：专辑资料管理器。按优先级问各个源。封面、年份、介绍各自取第一个得分够高的结果，所以图和介绍可以来自不同的源。三项都齐了就不再往下问。

文件 lib/data/album/album_text.dart。专辑介绍里的换行和简单HTML收成纯文本。

文件 lib/data/album/itunes_album_provider.dart。iTunes搜索里的专辑图和发行

年份。这个源没有专辑介绍。本文件定义了ItunesAlbumProvider。

文件 lib/data/album/netease_album_provider.dart。网易云专辑图、发行年份和专辑介绍。本文件定义了NeteaseAlbumProvider。

文件 lib/data/album/qq_album_provider.dart。QQ音乐专辑图、发行年份和专辑介绍。本文件定义了QqAlbumProvider。

文件 lib/data/audio/audio_cache_store.dart。音频缓存文件与索引(M3.2)文件布局:`<dir>/<cacheKey>.audio`(完整)与`.audio.tmp`(下载中)。索引写入`cache_index`,路径存相对dir的文件名,便于迁移缓存目录。本文件定义了AudioCacheStore。

文件 lib/data/audio/cache_download_manager.dart。下载任务登记处(M3.2)同一cacheKey只存在一个下载任务,播放与批量缓存共享;不同key互不阻塞。本文件定义了CacheDownloadManager。

文件 lib/data/audio/cache_download_task.dart。播放器能识别的媒体类型。B站CDN对DASH`.m4s`常返回`application/octet-stream`。macOS/iOS经just_audio本地代理播放时，地址没有扩展名，AVPlayer只根据Content-Type判断格式；非audio/video会报`(-11828)CannotOpen`。这时保留fallback（DASH音频为`au。本文件定义了CacheDownloadTask。CacheDownloadTask：单个音频的后台完整下载(M3.2)顺序写入`.tmp`,支持断点续传(Range);完成后finalize为正式文件。播放端通过read跟随已下载部分读取,实现边听边存。

文件 lib/data/audio/cache_migrator.dart。迁移进度(已处理文件数,总文件数)。本文件定义了CacheMigrator。CacheMigrator：缓存目录迁移(M3.5)`cache_index`只存文件名,迁移只需移动文件,无需改索引。跨盘符时rename失败,回退为复制后删除。

文件 lib/data/audio/caching_stream_audio_source.dart。边听边存音频源(M3.2,D5)1.`cache_index`完整命中:直接读本地文件,不发网络请求2.否则启动/复用CacheDownloadManager的顺序下载任务,范围请求从已下载部分读取,未到达的部分等待下载进度3.中断或切歌时`.tmp`保留,下次断点续传。本文件定义了CachingStreamAudioSource。

文件 lib/data/bili/audio_quality_selector.dart。一条DASH音频候选流。本文件定义了AudioQualitySelector。AudioQualitySelector：音质选择(M2.4)规则:`min(用户偏好,可用最高档)`;未登录上限132K(D2)。没有不高于目标档位的候选时,退而选可用的最低档。

文件 lib/data/bili/bili_login_api.dart。扫码状态(M7.1)。本文件定义了QrStatus、QrWaiting、QrScanned、QrExpired、QrConfirmed、BiliUser、BiliLoginApi、

HttpHeaders。QrWaiting：86101:未扫描。QrScanned：86090:已扫描,等待手机确认。QrExpired：86038:二维码已过期。QrConfirmed：0:登录成功,Cookie已写入SecureStore。

文件 lib/data/bili/bili_ranking_api.dart。音乐区热门榜。发现页在用户还没搜索时用它做推荐。本文件定义了MusicChartSource、MusicZone、MusicChartBundle、BiliRankingApi。MusicZone：排行榜v2里适合当歌曲听的音乐子分区。`tidv2`见分区一览。乐评、教学、歌单和饭拍不单列。MusicChartBundle：同一份音乐区排行拆出的推荐列表，顺序即展示顺序。BiliRankingApi：B站音乐区排行(rid=1003)。分区tid`3`是搜索用的音乐区，排行榜v2的音乐rid是`1003`。榜单按近期数据动态更新，用来代表当日播放热度高的音乐。

文件 lib/data/bili/bili_search_api.dart。Bilibili搜索API(M2.2)。本文件定义了BiliSearchApi。

文件 lib/data/bili/bili_stream_api.dart。流选择结果(M2.4)。本文件定义了BiliStreamApi。BiliStreamApi：Bilibili音频流地址API(M2.4)。

文件 lib/data/bili/bili_video_api.dart。Bilibili视频详情API(M2.3)。本文件定义了BiliVideoApi。

文件 lib/data/bili/models/search_result.dart。搜索结果(M2.2)。本文件定义了SearchResult、SearchPage。

文件 lib/data/db/app_database.dart。应用数据库，包含所有业务表。本文件定义了AppDatabase。

文件 lib/data/db/daos/cache_dao.dart。获取缓存条目。本文件定义了CacheDao。

文件 lib/data/db/daos/history_dao.dart。一条播放历史+对应曲目。本文件定义了HistoryDao。

文件 lib/data/db/daos/lyrics_dao.dart。歌词缓存索引(M5.3)。本文件定义了LyricsDao。

文件 lib/data/db/daos/playlists_dao.dart。获取所有歌单。本文件定义了PlaylistsDao。

文件 lib/data/db/daos/settings_dao.dart。获取设置值。本文件定义了SettingsDao。

文件 lib/data/db/daos/stream_urls_dao.dart。StreamUrlsDAO(M2.5)。本文件定义了StreamUrlsDao。

文件 lib/data/db/daos/tracks_dao.dart。根据bvid和cid查询轨道。本文件定义了TracksDao。

文件 lib/data/db/tables.dart。音频轨道表。本文件定义了Tracks、Playlists、PlaylistTracks、PlayHistory、LyricsCache、CacheIndex、StreamUrls、QueueSnapshot。Playlists：歌单表。PlaylistTracks：歌单-轨道关联表。PlayHistory：播放历史表。LyricsCache：歌词缓存表。

文件 lib/data/era/era_catalog.dart。某一年代里的一首歌，只描述曲目，不带播放地址。本文件定义了EraTrack、EraCatalog。EraCatalog：年代歌单。播放仍由B站搜索这些歌名来完成。

文件 lib/data/era/era_titles.dart。标题点名的年代。70、80、90这种跨年代合集会返回多个。

文件 lib/data/era/netease_era_catalog.dart。用网易云歌单提供某一年代的歌曲名和歌手。只收歌单标题能确定属于这一年代的歌单。有曲目数时选更长的，否则官方歌单优先。本文件定义了NeteaseEraCatalog。

文件 lib/data/lyrics/bili_subtitle_provider.dart。当前视频的B站CC字幕，转成LRC。字幕属于正在播放的这一P，命中后直接采用。本文件定义了BiliSubtitleProvider。

文件 lib/data/lyrics/kugou_lyrics_provider.dart。酷狗歌词。先搜歌曲哈希，再按哈希拉同步歌词。排在QQ音乐后面、酷我前面。本文件定义了KugouLyricsProvider。

文件 lib/data/lyrics/kuwo_lyrics_provider.dart。酷我歌词。搜索曲目后把逐行时间轴收成LRC。排在酷狗后面、LRCLIB前面。本文件定义了KuwoLyricsProvider。

文件 lib/data/lyrics/lrclib_provider.dart。LRCLIB歌词源(M5.2)`GEThttps://lrclib.net/api/search?track_name&artist_name`只取带同步歌词(`syncedLyrics`)的结果,用LyricsScorer打分并按得分降序返回。超时lyricsRequestTimeout，失败抛LyricsSourceUnavailable。本文件定义了LrclibProvider。

文件 lib/data/lyrics/lyrics_http.dart。每个歌词请求的连接和读取上限。失败后由管理器再试一次。

文件 lib/data/lyrics/lyrics_manager.dart。歌词查询结果(M5.4)。本文件定义了LyricsResult、LyricsFound、LyricsUncertain、LyricsNone、LyricsManager。LyricsFound：已匹配:lyrics.offset已合并LRC自带offset与用户校准偏移。LyricsUncertain：得分在0.60-0.80之间,需要用户确认。LyricsNone：无歌词(用户标记,或所有候选得分过低)。LyricsManager：歌词管理器(M5.3/M5.4)1.查`lyrics_cache`:命中则读本地LRC,`no_lyrics`直接返回无歌词2.否则按优先级依次查询远程提供者,首个≥autoThreshold即采用并缓存3.同一曲目的并发查询合并。

文件 lib/data/lyrics/netease_lyrics_provider.dart。网易云歌词。搜索曲目后只拉取得分最高的几首同步歌词。优先级排在B站字幕和LRCLIB前面。得分够了就用网易云，不再继续问后面的源。本文件定义了NeteaseLyricsProvider。

文件 lib/data/lyrics/qq_lyrics_provider.dart。QQ音乐歌词。搜索曲目后只拉取得分最高的几首同步歌词。排在B站字幕后面、LRCLIB前面。本文件定义了QqLyricsProvider。

文件 lib/data/player/audio_engine.dart。播放引擎抽象(M3.3)隔离just_audio的平台依赖,让BiliAudioHandler可以用Fake引擎单测。桌面端(Windows/Linux)由`just_audio_media_kit`提供实现(D4),接口不变。本文件定义了AudioEngine、JustAudioEngine。JustAudioEngine：基于just_audioAudioPlayer的实现(M3.3)。

文件 lib/data/player/bili_audio_handler.dart。播放历史记录钩子(M6.5):开始播放一首时调用,返回更新进度的回调。本文件定义了BiliAudioHandler。BiliAudioHandler：后台播放入口(M3.1/M3.4)-播放器每次只加载当前曲;下一首仅预解析流地址(D8)-播放模式由PlaybackQueue控制,引擎始终不循环-队列变化后防抖写入`queue_snapshot`。

文件 lib/data/player/media_library.dart。一次浏览树点播：整表播放，或跳到当前队列的下标。本文件定义了BrowsePlay、MediaLibrary。MediaLibrary：AndroidAuto/车机媒体浏览树。只暴露正在播放、本地歌单、最近播放。车机用系统界面，不嵌入Flutter。

文件 lib/data/player/queue_snapshot_store.dart。队列快照持久化(M3.4)写入单行`queue_snapshot`;scheduleSave500ms防抖,避免频繁写库。本文件定义了QueueSnapshotStore。

文件 lib/data/player/sleep_timer.dart。定时停止状态(M3.6)。本文件定义了SleepState、SleepOff、SleepCountdown、SleepAfterTracks、SleepTimer。SleepCountdown：倒计时中。SleepAfterTracks：播完tracksLeft首后停止(1=播完当前)。SleepTimer：定时停止(M3.6):15/30/60分钟、播完当前/下一首。

文件 lib/data/repositories/bili_repositories.dart。基于B站接口的搜索仓储(M2.6)。本文件定义了BiliSearchRepository、BiliStreamRepository。BiliStreamRepository：基于StreamUrlManager的流地址仓储(M2.6)。

文件 lib/data/repositories/bili_video_repository.dart。基于B站接口+drift的视频仓储(M2.6)。本文件定义了BiliVideoRepository。

文件 lib/data/repositories/fake_repositories.dart。内置演示数据(M2.6):供

M6UI在接口就绪前开发,不发任何网络请求。本文件定义了FakeSearchRepository、FakeVideoRepository、FakeStreamRepository。

文件 lib/data/repositories/repositories.dart。搜索结果+标题清洗结果,供UI直接展示(M2.6)。本文件定义了SearchRepository、DiscoverGroup、DiscoverTab、DiscoverCharts、VideoRepository、StreamRepository。DiscoverGroup：发现页上的两档推荐。声明顺序就是芯片从左到右的顺序。DiscoverTab：发现页推荐分类。DiscoverCharts：一次拉榜得到的热度、评价和排行榜分区。曲目都已经按15分钟规则展开。年代不在这里，见SearchRepository.eraChart。

文件 lib/data/repositories/stream_url_manager.dart。流地址管理器(M2.5)-读写`stream_urls`表,`expiresAt-10min`内视为过期-同一(bvid,cid,quality)的并发请求合并为一次-refresh用于CDN403等场景强制刷新。本文件定义了StreamUrlManager。

文件 lib/data/update/app_release.dart。GitHubRelease里的一个附件。本文件定义了ReleaseAsset、AppRelease。AppRelease：一次可用于升级的正式Release。

文件 lib/data/update/app_version_source.dart。读取当前安装的版本。本文件定义了AppVersionSource、FixedAppVersionSource、PackageAppVersionSource。FixedAppVersionSource：固定版本。测试，以及还没注入真实来源时使用。PackageAppVersionSource：从平台包信息读取pubspec写入的版本。

文件 lib/data/update/host_platform.dart。当前设备在升级逻辑里使用的系统名和CPU。本文件定义了UpdateHost。

文件 lib/data/update/local_update_installer.dart。把安装包下载到本机并打开。本文件定义了UpdateInstaller、NoopUpdateInstaller、InstallPermissionException、InstallLaunchException、LocalUpdateInstaller。NoopUpdateInstaller：不下载。测试默认实现。InstallPermissionException：系统没有允许本应用安装未知来源的APK。InstallLaunchException：下载或打开安装包失败。LocalUpdateInstaller：下载Release附件到临时目录，再按扩展名打开。macOS由沙盒里的本应用下载磁盘映像时，系统会写上`com.apple.quarantine`，并把来源标成AppSandbox。拖进「应用程序」的副本因此被拒绝启动。安装时改在沙盒外替换正在运行的应用包，并去掉隔离标记。

文件 lib/data/update/release_source.dart。读取可用于升级的正式版本。本文件定义了ReleaseSource、EmptyReleaseSource、UpdateCheckException、GithubReleaseSource。EmptyReleaseSource：不访问网络。测试和未注入真实源时使用。UpdateCheckException：检查更新失败。GithubReleaseSource：从GitHubReleases读取可以升级到的版本。请求最近100条Release，跳过草稿和预发布（例如`continuous`），

在剩下的版本里取最高的一个。推送到main的构建会发布`vX.Y.Z+N`，构建号每次加一，因此高于已经安装的包。

23 界面交互

负责搜索、播放、歌单、队列、歌词和设置这些用户能看到的页面。

文件 lib/features/album/album_controller.dart。专辑资料管理器。未注入时播放页继续用B站封面。本文件定义了AlbumController。AlbumController：当前曲目的官方专辑图、年份和介绍。切歌时重新查询。

文件 lib/features/common/add_to_playlist_sheet.dart。选择歌单并加入(M6.4);支持就地新建。

文件 lib/features/common/cover_image.dart。封面图;缺失或加载失败时显示音符占位。本文件定义了CoverImage。

文件 lib/features/history/history_view.dart。历史分组:今天/昨天/本周/更早。本文件定义了HistoryGroup、HistoryView。HistoryView：播放历史(M6.5)。

文件 lib/features/home/bottom_nav.dart。底栏与侧栏共用的选中项。本文件定义了BottomNav、NavTarget。BottomNav：底部导航栏(M6):自动跳转shell路由。NavTarget：一个导航项：图标和文字共用同一块悬停、按下和选中区域。

文件 lib/features/home/home_shell.dart。主界面外壳：按内容区宽高切换底栏、侧栏或主从栏。本文件定义了HomeShell。

文件 lib/features/library/library_page.dart。歌单库(M6.4):本地歌单/播放历史。本文件定义了LibraryPage。

文件 lib/features/library/playlist_detail_page.dart。歌单详情(M6.4):播放全部/随机,拖拽排序,滑动删除。本文件定义了PlaylistDetailPage。

文件 lib/features/login/login_controller.dart。登录API(M7.2);bootstrap中override。本文件定义了LoginController。LoginController：当前用户(M7.2):启动时加载,登录后刷新。

文件 lib/features/login/login_cookie_source.dart。从本机网页容器读取登录Cookie。凭证不离开这台设备。本文件定义了LoginCookieSource、PlatformLoginCookieSource。

文件 lib/features/login/login_method.dart。Android/iOS走网页登录。桌面端仍用扫码。本文件定义了LoginMethod。

文件 lib/features/login/login_page.dart。登录页。手机默认网页登录，桌面默认扫码。本文件定义了LoginPage、LoginPrivacyConsent、LoginMethodBody。

LoginPrivacyConsent：登录页底部的隐私政策勾选。默认不勾选。

文件 lib/features/login/login_web.dart。不带WebView标记，避免登录页被当成应用内置浏览器而收起短信和密码。

文件 lib/features/login/qr_login_panel.dart。扫码登录(M7.2)。桌面端默认使用；手机上作为备选。本文件定义了QrLoginPanel。

文件 lib/features/login/web_login_panel.dart。打开哔哩哔哩登录页，从本机网页容器读取登录凭证。本文件定义了WebLoginPanel。

文件 lib/features/lyrics/lyrics_controller.dart。歌词管理器;bootstrap中override(M5)。本文件定义了LyricsController。LyricsController：当前曲目的歌词状态(M5.5)切歌时自动重新查询;偏移调整/手动选择后就地更新,无需重新搜索。

文件 lib/features/lyrics/lyrics_view.dart。歌词区(M5.5/M6.2):当前行高亮居中,点击行跳转。本文件定义了LyricsView。

文件 lib/features/player/player_actions.dart。UI共用的播放/歌单操作(M6),失败时返回Result.failure由页面提示。本文件定义了PlayerActions。

文件 lib/features/player/player_page.dart。播放页(M6.2)。本文件定义了PlayerPage。

文件 lib/features/player/widgets/car_player.dart。车机播放界面。顶部放大当前歌词。左侧是完整歌词或播放列表，右侧是封面和介绍。播放控制在最底部。歌词和队列的切换半透明，浮在左侧区域左上角。本文件定义了CarPlayer。

文件 lib/features/player/widgets/mini_player.dart。底部常驻迷你播放栏(M0.4/M6.2);无播放内容时隐藏。本文件定义了MiniPlayer。

文件 lib/features/player/widgets/now_playing_pane.dart。大宽度常驻播放栏，和全屏播放页共用同一套控件与播放器。本文件定义了NowPlayingPane。

文件 lib/features/player/widgets/player_transport.dart。播放页和右侧播放栏共用的封面与标题。匹配到官方专辑图时用它替换B站封面，并在标题下显示年份和专辑介绍。本文件定义了PlayerCoverSection、CurrentLyricLine、PlayerProgressBar、PlayerControls、PlayerSleepIndicator。CurrentLyricLine：当前歌词一行。没有歌词时不占内容。

文件 lib/features/queue/queue_page.dart。播放队列(M6.3)顺序模式可拖拽排序;随机模式禁用拖拽。可移除任意单项，含正在播放的。可按歌名、歌手或UP主在队列内搜索，点结果从那一首开始播放。本文件定义了QueuePage、QueueList。QueueList：队列列表。大宽度播放栏内嵌时不带页面脚手架。非upcomingOnly时顶部可搜索。搜索时不能

拖拽，点选仍按原队列位置跳转。upcomingOnly为车机布局：只显示接下来最多maxItems首，不能拖拽、不能搜索。

文件 lib/features/search/search_controller.dart。搜索页状态(M6.1)。本文件定义了SearchState、SearchController、SearchHistory。SearchHistory：搜索历史:最近在前,最多20条,存settings表。

文件 lib/features/search/search_page.dart。发现页(M6.1)还没搜索时展示音乐区推荐，可在排行榜分区和年代之间切换。本文件定义了SearchPage。

文件 lib/features/settings/cache_page.dart。缓存管理占位(M6.7,完整界面见v0.2.3)。本文件定义了CachePage。

文件 lib/features/settings/settings_page.dart。设置(M6.6)。本文件定义了SettingsPage。

文件 lib/features/settings/update_controller.dart。一次升级检查或安装所处的阶段。本文件定义了UpdatePhase、UpdateFailure、UpdateState、UpdateController。UpdateFailure：失败发生在哪一步。UpdateState：升级界面使用的状态。UpdateController：检查GitHub正式版，并在有安装包时下载打开。

文件 lib/features/settings/update_prompt.dart。进入主界面后检查一次更新。车机布局下不弹窗。本文件定义了UpdatePrompt、UpdateSettingsTile。UpdateSettingsTile：设置页里的升级入口。

24 界面文案

负责界面上使用的中文文案。

文件 lib/l10n/l10n.dart。将领域错误转为用户可读文案。本文件定义了L10nX。

本设计说明书说明音赴光年 V0.1.0 的结构、数据、播放、歌词、缓存和歌单。界面分档、五种播放模式、音质档和本机各张数据表都以源程序为准。程序鉴别材料摘录的是同一套源程序的开头连续 30 页和结尾连续 30 页。软件名称为音赴光年，版本号为V0.1.0。

音赴光年只播放音频，不播放画面。

歌单、歌词偏移、播放队列和登录凭据都保存在本机。

随机播放在点下一首时才从本轮未听曲目中选取。

已加入歌单的缓存曲目不参与空间不足时的清理。

未登录时音质最高到 132K，音质档另有 64K、192K、Dolby 和 Hi-Res。

窗口宽度达到 1194 且高度至少 500 时，右侧常驻正在播放。

关闭桌面窗口后程序留在系统托盘中，再次打开不自动出声。